Orange problem-4

Name: Samarth A SRN: PES1UG24AM243

REPO LINK: https://github.com/samarth2006-28/PES1UG24AM243-pes-vcs?tab=readme-ov-file#submission-requirements

Run Phase 1 Tests (Screenshot 1A):

```
samarth@SAMARTH28:/mnt/c/Users/samar/OneDrive/Desktop/os-u4-orange-problem-main$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
samarth@SAMARTH28:/mnt/c/Users/samar/OneDrive/Desktop/os-u4-orange-problem-main$
```

Screenshot 2: Phase 2 Tests:

```
# Run the tree test
./test_tree
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function 'write_tree_recursive':
```

```
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Staging Files:

```
samarth@SAMARTH28:/mnt/c/Users/samar/OneDrive/Desktop/os-u4-orange-problem-main$ ./pes init
./pes add README.md report.md
./pes status
Initialized empty PES repository in .pes/
Segmentation fault (core dumped)
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  report.md
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
```

Log with 3 commits:

```
samarth@SAMARTH28:/mnt/c/Users/samar/OneDrive/Desktop/os-u4-orange-problem-main$ # Screenshot 4A: Log with three commits
# (Note: You must set the author first)
export PES_AUTHOR="Samarth <PES1UG24AM243>"
./pes commit -m "Commit 1: Setup"
./pes commit -m "Commit 2: Added documentation"
./pes commit -m "Commit 3: Final core logic"
./pes log

# Screenshot 4B: Object Growth
find .pes -type f | sort

# Screenshot 4C: HEAD and Refs
cat .pes/refs/heads/main
cat .pes/HEAD
Committed: 2f8631b000aa... Commit 1: Setup
Committed: 8151b472b208... Commit 2: Added documentation
Committed: 173eac26c354... Commit 3: Final core logic
commit 173eac26c354e2effe9101f42bdb782452f812b2215f77d5e5c2a8166a29d56b
Author: Samarth <PES1UG24AM243>
Date:   1776317997

    Commit 3: Final core logicver auth!

commit 8151b472b20880ff18a664d5480665b122702025e920b2d14666c774be2c29c5
Author: Samarth <PES1UG24AM243>
Date:   1776317996

    Commit 2: Added documentation auth!

commit 2f8631b000aad0ce80c2048e004ace994ce9afe976db2acdc1d5e4a2ce67fd80
Author: Samarth <PES1UG24AM243>
Date:   1776317996

    Commit 1: Setupit 1: SetupQ

.pes/HEAD
.pes/index
.pes/objects/17/3eac26c354e2effe9101f42bdb782452f812b2215f77d5e5c2a8166a29d56b
.pes/objects/2f/8631b000aad0ce80c2048e004ace994ce9afe976db2acdc1d5e4a2ce67fd80
.pes/objects/60/459622a6a99c40ddfcd668e7cbd0d70a4d53aeb448a251aa982ea2043e3838
.pes/objects/77/bfdac4908bf74e05b50cdfb77589d6956adb289a3cc410ac9a3c4e3a7e887a
.pes/objects/81/51b472b20880ff18a664d5480665b122702025e920b2d14666c774be2c29c5
.pes/objects/fd/e45ce5081a6cd515da65b2cc1ae9be1d1658614dfaf11b20b6e30b4fe59427
.pes/refs/heads/main
173eac26c354e2effe9101f42bdb782452f812b2215f77d5e5c2a8166a29d56b
ref: refs/heads/main
```

Run Integration Test:

```
samarth@SAMARTH28:/mnt/c/Users/samar/OneDrive/Desktop/os-u4-orange-problem-main$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
test_sequence.sh: line 36:  1395 Segmentation fault      (core dumped) $PES add file.txt hello.txt
make: *** [Makefile:45: test-integration] Error 139
```

Clean up between runs

```
samarth@SAMARTH28:/mnt/c/Users/samar/OneDrive/Desktop/os-u4-orange-problem-main$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
```

Final Integration Screenshot

```
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: b0a79875477b... Initial commit

Log after first commit:
commit b0a79875477b1aa5c073dbf21832cc2e0338a6636e3acb3966bf1068e6000cea
Author: Samarth <PES1UG24AM243>
Date:   1776317065

    Initial commit


--- Second Commit ---
Committed: 1606a6694463... Update file.txt

--- Third Commit ---
Committed: 7280e8f37572... Add farewell

--- Full History ---
commit 7280e8f3757252eb6c346bfeede2ec44d195c3af1ecf2d8a6cc4dfc1d6dfe70d
Author: Samarth <PES1UG24AM243>
Date:   1776317065

    Add farewell

commit 1606a66944631024334ad89726dbcb98cb3184354afb47b5414ba72ed6123404
Author: Samarth <PES1UG24AM243>
Date:   1776317065

    Update file.txt

commit b0a79875477b1aa5c073dbf21832cc2e0338a6636e3acb3966bf1068e6000cea
Author: Samarth <PES1UG24AM243>
Date:   1776317065

    Initial committial commitEQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
7280e8f3757252eb6c346bfeede2ec44d195c3af1ecf2d8a6cc4dfc1d6dfe70d

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/16/06a66944631024334ad89726dbcb98cb3184354afb47b5414ba72ed6123404
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/72/80e8f3757252eb6c346bfeede2ec44d195c3af1ecf2d8a6cc4dfc1d6dfe70d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b0/a79875477b1aa5c073dbf21832cc2e0338a6636e3acb3966bf1068e6000cea
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Phase 5: Branching and Checkout

Q5.1: Outline the logical steps required for pes checkout <branch>. Why is it more complex than just updating the .pes/HEAD file?

- **Logical Steps**:

    1. Read the target branch's latest commit hash from .pes/refs/heads/<branch>.

2. Extract the root tree hash associated with that commit.
3. Traverse the tree recursively and overwrite the current files in the working directory with the versions from the object store.
4. Update the .pes/index to match the new state.
5. Finally, update .pes/HEAD to point to the new branch.

- **Complexity**: It is complex because you must handle "dirty" files—changes the user has made but not committed. If a checkout overwrites these changes, they are lost forever. A robust system must detect these conflicts and warn the user before switching.

Q5.2: How would you detect if the working directory is "dirty" (has uncommitted changes) before allowing a checkout?

- **Detection Method**: You compare the metadata (mtime and size) and the SHA-256 hash of every file in the current folder against the entries in the .pes/index. If any file in the folder differs from the index, or if the index differs from the HEAD commit, the directory is considered "dirty."

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

- **Committing in Detached HEAD**: When you commit while in a detached HEAD state, the new commit is created as usual, but no branch pointer (like main) is updated to track it. HEAD simply moves to the new commit hash. If you switch away to another branch, you lose the reference to these new commits because nothing points to them.
- **Recovery**: A user can recover these commits by checking the **Reflog** (if implemented) to find the lost hash, or by manually creating a new branch that points to that specific commit hash before the Garbage Collector deletes it.

Phase 6: Garbage Collection (GC)

Q6.1: Propose an algorithm for pes gc. How would you determine if an object is "reachable"?

- **Algorithm (Mark & Sweep)**:
    1. **Mark Phase**: Create a set of "reachable" objects. Start from all "roots" (every branch in refs/heads and the HEAD). Recursively traverse every commit and every tree/blob they point to, adding their hashes to the reachable set.
    2. **Sweep Phase**: Iterate through all files in the .pes/objects/ directory. If a file's hash is NOT in the "reachable" set, delete it safely.
- **Determining Reachability**: An object is reachable if it is part of the history of any active branch. If a commit or blob is no longer pointed to by any branch or tag (an "orphan"), it is no longer reachable.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

- **The Danger (Race Condition)**: Imagine a commit operation is currently writing a new "blob" object but hasn't yet created the "commit" object that points to it. If the Garbage Collector runs at that exact moment, it will see the new blob as "unreachable" (orphan) because no commit or branch points to it yet. The GC might delete the blob just before the commit operation tries to link it, resulting in a broken, corrupted commit.
- **Git's Solution**: Git uses a **"grace period"** (usually 2 weeks by default). The GC will only delete objects that are both unreachable **and** older than a certain time (the "prune" threshold). Since a concurrent commit would be writing new objects, their creation timestamp would be very recent, and the GC would skip them even if they appear "unreachable" for a split second.